PatchPilot

An open-model agent that fixes real bugs, with trained fault localization and sandboxed verification

Technical report

Tejaswini Prabhakaran

Built on 28 October 2026 from commit 7801983

https://github.com/Tejaswiniprabhakaran/patchpilot

Contents

Abstract 4
Chapter 1 — Introduction 5
1.1 Background 5
1.2 Problem statement 5
1.3 Motivation 5
1.4 Objectives 5
1.5 Scope 5
1.6 Research questions 5
Chapter 2 — Literature Survey 7
2.1 Automated program repair 7
2.2 LLM coding agents and SWE-bench 7
2.3 Fault localization 7
2.4 Parameter-efficient fine-tuning 7
2.5 Comparison of related work 7
2.6 Research gap 8
Chapter 3 — System Requirements 9
3.1 Hardware (development machine, measured) 9
3.2 Software 9
3.3 Functional requirements 9
3.4 Non-functional requirements 9
Chapter 4 — System Design 10
4.1 Architecture 10
4.2 Modules 11
4.3 The agent loop 12
4.4 Software / AI split 12
Chapter 5 — Implementation 13
5.1 Interfaces 13
5.2 The GitHub bot 18
Chapter 6 — Datasets and Model Training 20
6.1 Evaluation data 20
6.2 Training data and leakage check 20
6.3 Training setup 20
6.4 Ranker training run 20
Chapter 7 — Results and Discussion 21
7.1 Benchmark validity (`results/sanity_summary.csv`) 21

7.2 Inference cost on the laptop (`results/llm_speed_cpu.jsonl`) 21
7.3 Fault localization (RQ2, part 1) 21
7.4 Resolve rates and paired tests (RQ1, RQ2, RQ4) 22
7.5 Runs re-run by fixed rules (D13, D17) 23
7.6 Cost per instance 24
7.7 Failure analysis (RQ5) 24
7.8 Answers to the research questions 24
7.9 Threats to validity 24
Chapter 8 — Testing 26
Chapter 9 — Conclusion and Future Work 27
References 28
Appendix 30

Abstract

Automated program repair aims to produce correct patches for real bugs with little human effort. Recent large language model (LLM) agents can resolve some real GitHub issues, but most rely on large proprietary models, localize faults with ad-hoc search and report a single resolve rate without a careful account of why they fail. **PatchPilot** is an end-to-end repair system built around an open model that runs on a laptop CPU (Google's Gemma 4 E4B, served by Ollama), a *trained* fault-localization ranker (a UniXcoder cross-encoder that re-ranks BM25 candidates) and *sandboxed verification*: every candidate patch is tested inside a network-isolated Docker container, and failing attempts are retried with the test output as feedback. The system includes a command-line tool, a web dashboard, a GitHub bot that comments verified fixes on failing pull requests, and an evaluation harness on QuixBugs and a seeded subset of SWE-bench Lite with paired McNemar tests and an automatic failure taxonomy. Training data come from the SWE-bench training split, with a leakage check that finds zero overlap with the evaluation set.

Results. With base Gemma 4 E4B on a laptop CPU, the baseline (B0, LLM-search localization) resolved 9/16 (56.2%; run still in progress) on QuixBugs. Resolve rates with the trained ranker (E1): TODO: not yet run. Fault-localization accuracy of the trained ranker: TODO: not yet run. The QLoRA patch-generator pipeline (training data, notebook and model card) is complete but was not trained, because it needs a GPU, so the effect of fine-tuning (RQ3) remains open.

Chapter 1 — Introduction

1.1 Background

Software defects are expensive to find and fix. Automated program repair (APR) generates patches automatically, usually guided by tests. Early search-based tools such as GenProg evolved program variants until the tests passed [1]; learning-based tools translated buggy lines into fixed ones [2]; and pre-trained code models enabled repair without task-specific training [3]. Large language models changed the field again: conversational repair with test feedback fixes many classical benchmark bugs cheaply [4], and agents that browse a repository, edit files and run tests can resolve a fraction of real GitHub issues in SWE-bench [5], [6], [7], [8].

1.2 Problem statement

Given a repository with a failing test (and, for real issues, a bug report), produce a patch to the source code that makes the target tests pass without breaking previously passing tests — and explain, with evidence, how often this succeeds and why it fails when it does not.

1.3 Motivation

Most published agents use large proprietary models and treat localization as free-form search by the model. A student or small team cannot run them locally, and their results say little about which component matters. PatchPilot is built to answer that question with an open model that runs on a laptop, a localization model that is trained rather than prompted, and a fully logged, reproducible evaluation.

1.4 Objectives

1. A safe execution environment that runs untrusted repository code with no network and bounded resources.
2. An agent with a small, auditable tool interface and test-feedback retries, recording every step.
3. A trained fault-localization ranker compared with BM25 and embedding retrieval.
4. A QLoRA-fine-tuned Gemma 4 E4B patch generator compared with the base model.
5. A rigorous evaluation with baselines, ablations and paired statistical tests, plus a failure taxonomy.
6. Product interfaces: CLI, dashboard and a GitHub pull-request bot.

1.5 Scope

Python programs only; evaluation on QuixBugs (40 programs) and a seeded 10-instance subset of SWE-bench Lite drawn from 49 sanity-checked instances. All inference and the ranker's training run on a laptop CPU (decisions D12 and D16 in `docs/decisions.md`). The patch generator's QLoRA training (objective 4) is prepared for a free Kaggle GPU but was not run, so the comparisons that need the fine-tuned model stay open.

1.6 Research questions

	Question
RQ1	How many real bugs can an open-weight LLM agent fix on QuixBugs and the SWE-bench Lite subset?
RQ2	How much does a trained fault-localization model improve the fix rate over LLM-based search?
RQ3	How much does LoRA fine-tuning the patch generator help over the base model?
RQ4	How much does test-feedback retrying help, and how many

	attempts are worth it?
RQ5	Where does the agent fail?

Chapter 2 — Literature Survey

2.1 Automated program repair

GenProg uses genetic programming over the program's own statements, with the test suite as the fitness function [1]. SequenceR frames repair as sequence-to-sequence translation of the buggy line, trained on commits [2]. AlphaRepair showed that a pre-trained code model (CodeBERT) can repair bugs "zero-shot" by filling in a masked buggy line, without training on historical fixes [3]. ChatRepair turned repair into a conversation with ChatGPT, feeding failing-test information back to the model after each wrong patch [4] — the same feedback idea PatchPilot uses.

2.2 LLM coding agents and SWE-bench

SWE-bench collects 2,294 real issues from 12 Python repositories, each with the tests that the developers' fix made pass [5]. SWE-agent designs an agent-computer interface (file viewer, search, editor) that lets a language model act in a repository [6]. AutoCodeRover searches the program's structure (classes and methods) and can use spectrum-based fault localization when tests exist [7]. Agentless argues that a fixed pipeline — localize, repair, validate — can match more complex agents at lower cost [8]; PatchPilot's agent is a pipeline of the same kind, with a trained localizer.

2.3 Fault localization

Spectrum-based fault localization ranks program elements by how often they are executed by failing versus passing tests [11]. DeepFL combines suspiciousness, fault-proneness and textual-similarity features in a deep model [12]. Information-retrieval localization ranks files by textual similarity to the bug report; BM25 [19] is the standard baseline. Pre-trained code encoders such as CodeBERT [13] and UniXcoder [14] can be fine-tuned as *cross-encoders* that read the report and a code candidate together, an approach that re-ranks retrieval shortlists well in text search. Dense retrieval with vector indexes such as FAISS [17] is the usual alternative.

2.4 Parameter-efficient fine-tuning

LoRA trains small low-rank matrices next to frozen weights [15]; QLoRA does so on a 4-bit quantized base model, making fine-tuning of multi-billion-parameter models possible on a single GPU [16].

2.5 Comparison of related work

#	Work	Year	Repair approach	Localization	Model	Verification	Evaluation
[1]	GenProg	2012	genetic programming	test coverage	none	test suite	C programs
[2]	SequenceR	2019	seq2seq on buggy line	given (perfect)	trained seq2seq	tests	Defects4J
[3]	AlphaRepair	2022	cloze-style infill	given	CodeBERT (zero-shot)	tests	Defects4J, QuixBugs
[4]	ChatRepair	2024	conversational, test feedback	given	ChatGPT	tests	Defects4J, QuixBugs
[5]	SWE-bench	2024	benchmark	—	various	hidden tests	2,294 GitHub issues
[6]	SWE-agent	2024	agent with custom interface	model-driven search	GPT-4 class	tests	SWE-bench
[7]	AutoCodeRover	2024	agent, AST-aware search	code search + SBFL	GPT-4 class	tests	SWE-bench Lite
[8]	Agentless	2025	localize-repair-validate	hierarchical, model-driven	GPT-4o class	reproduction tests	SWE-bench Lite

			pipeline				
[9]	QuixBugs	2017	benchmark	—	—	tests	40 programs
[10]	Defects4J	2014	benchmark	—	—	tests	Java faults
[11]	Spectrum-based FL	2007	—	coverage spectra	none	—	Siemens set
[12]	DeepFL	2019	—	learned from many features	deep network	—	Defects4J
[13]	CodeBERT	2020	—	(encoder)	pre-trained bimodal	—	code search
[14]	UniXcoder	2022	—	(encoder)	pre-trained, AST-aware	—	code search, more
[16]	QLoRA	2023	—	—	4-bit + LoRA fine-tuning	—	instruction tuning
—	**PatchPilot**	2026	pipeline agent, test-feedback retries	**trained cross-encoder** over BM25	**open Gemma 4 E4B on a CPU** (QLoRA prepared)	network-off Docker sandbox, official harness	QuixBugs + SWE-bench Lite subset, McNemar tests, failure taxonomy

2.6 Research gap

Existing agents mostly (i) rely on large proprietary models, (ii) localize with prompting rather than a trained, evaluated ranker, and (iii) report a single resolve rate without paired statistics or a systematic account of failures. PatchPilot isolates the contribution of a trained localizer, of retries and (when trained) of fine-tuning, with controlled ablations on an open model small enough for a laptop.

Chapter 3 — System Requirements

3.1 Hardware (development machine, measured)

Item	Value
CPU / GPU	Intel Core i5-1335U (10 cores, 12 threads) / Intel Iris Xe, no CUDA GPU
RAM	15.6 GB, shared with Docker Desktop and the model server (about 8 GB for Gemma 4 E4B)
Disk	Docker data on a separate drive; PatchPilot kept under 10 GB (decision D14)
Training	ranker: the same laptop CPU (D16); patch generator: notebook prepared for a free Kaggle T4 GPU (not run)

3.2 Software

Python 3.11; Docker Desktop (Linux containers); Ollama (Gemma 4 E4B, Q4_K_M, 32K-token context, D17); the official SWE-bench harness 4.1.0; transformers (ranker training), peft, trl and bitsandbytes (patch-generator notebook); rank-bm25, FAISS and sentence-transformers (retrieval); FastAPI, Jinja2 and Pygments (dashboard); Typer and Rich (CLI); pytest, ruff and mypy (quality); GitHub Actions (CI and the bot); python-docx and Microsoft Word (this report).

3.3 Functional requirements

1. Run any benchmark instance's tests in an isolated container and report per-test results.
2. Given a failing instance, produce and verify a patch; record every step.
3. Localize faults with BM25, embeddings or the trained ranker, selectable by configuration.
4. Run experiments from a YAML file, resumably, saving raw results and trajectories.
5. Compute resolve rates, confidence intervals, paired tests, figures and a failure taxonomy.
6. Show runs, trajectories, diffs and metrics in a web dashboard; run a live demo.
7. Comment a proposed fix on a pull request whose tests fail, without pushing or merging.

3.4 Non-functional requirements

Security (no network for untrusted code, bounded CPU/memory/processes, no host mounts); reproducibility (pinned dataset revisions, seeds, configs saved with results); honesty (every number traceable to `results/`, and this report's result sections are generated from those files); resource frugality (one heavy job at a time, below-normal priority, never touching other projects' containers); maintainability (strict typing, unit tests, CI on every push).

Chapter 4 — System Design

4.1 Architecture

The central rule: repository code only runs inside the sandbox; every other component talks to it through a small API. The diagrams below are rendered from the Mermaid source in `docs/architecture.md`.

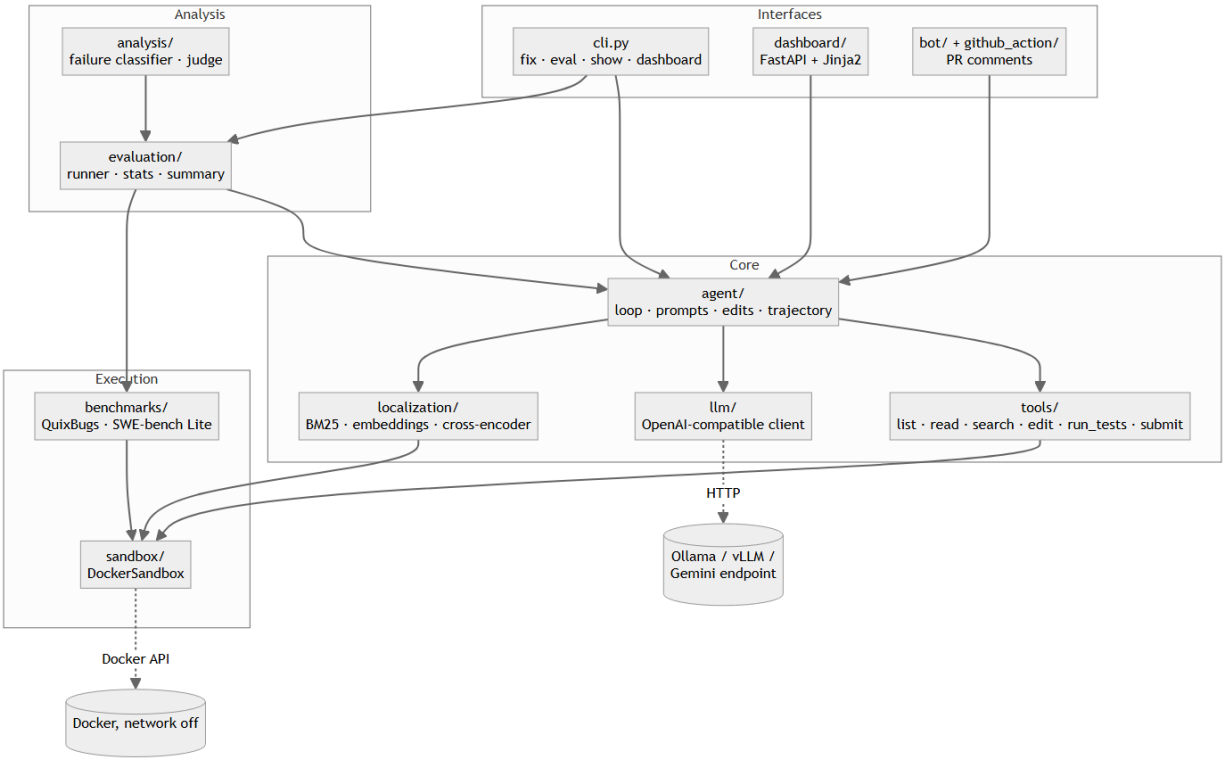


Figure 1. Architecture: modules, the sandbox and the model server

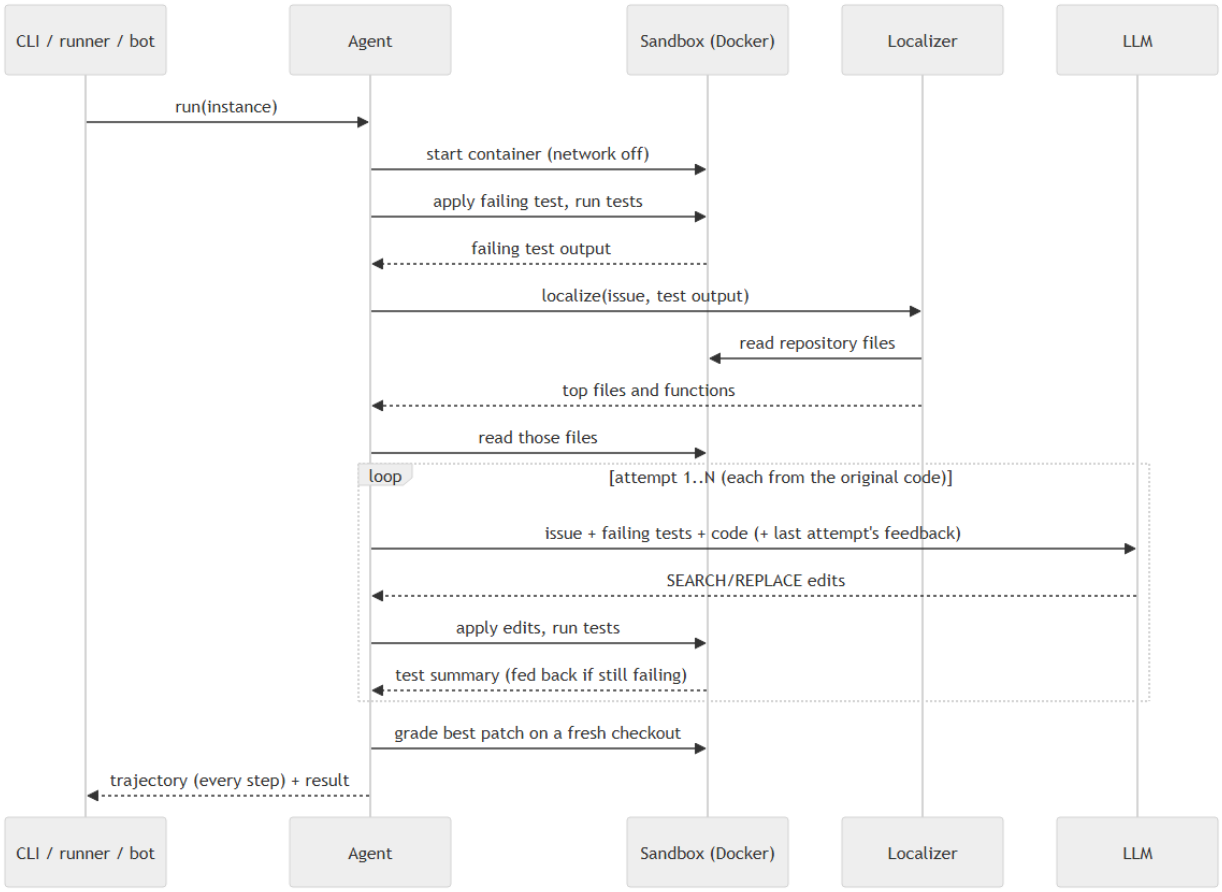


Figure 2. One agent run: reproduce, localize, repair with test feedback, grade independently

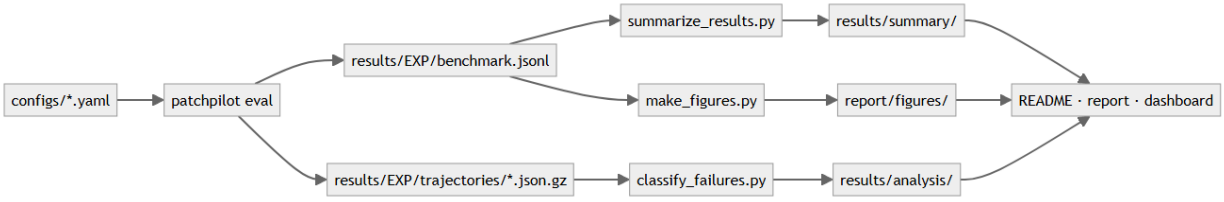


Figure 3. Experiment data flow from configurations to the README, the report and the dashboard

4.2 Modules

Module	Half	Responsibility
`sandbox`	S1	container per instance, no network, limits, safe patching, per-test parsing
`benchmarks`	S1	instances, resolution rule, QuixBugs and SWE-bench Lite, sanity check
`tools`, `agent`, `llm`	S2	six tools, retry loop, SEARCH/REPLACE edits, trajectories, model client
`cli`	S3	`fix`, `eval`, `show`, `dashboard`
`bot`, `github_action`	S4	pull-request bot
`dashboard`	S5	runs, trajectories, metrics, live demo
tests, CI, typing, pipeline	S6	quality and unattended runs
`localization`	A1, A3	labels, BM25, embeddings, cross-encoder, ranker localizer
`patchgen` + notebook 02	A2	fine-tuning data and QLoRA training
`evaluation`	A4	runner, scoring rules, statistics, tables, figures
`analysis`	A5	failure classifier, LLM judge, agreement
cards, `results/`	A6	model and dataset cards, raw results

4.3 The agent loop

1. Reset the checkout and apply the failing test; run the tests to capture the failure.
2. Localize: BM25 over all Python files, then the ranker (or, in the baseline, model-proposed keywords and code search) selects the top three files and their most suspicious functions.
3. Show the model the bug report, the failing tests and the trimmed files; parse its SEARCH/REPLACE blocks; apply them (each must match exactly once; edits that break compilation are undone).
4. Run the tests. If they still fail, start the next attempt from the original code with the previous edit and its test output as feedback, up to N attempts.
5. Submit the best candidate and grade it independently on a fresh checkout with the benchmark's official rule: all target tests pass and no previously passing test breaks.

4.4 Software / AI split

Software half	AI half
S1 sandbox · S2 agent framework · S3 CLI · S4 GitHub bot · S5 dashboard · S6 quality	A1 trained ranker · A2 QLoRA patch model · A3 retrieval · A4 evaluation · A5 failure analysis · A6 cards and tracking

Chapter 5 — Implementation

Each module's design, rationale and rejected alternatives are written up in `docs/explained/` (sandbox, benchmarks, agent, localization, patch generator, failure analysis, GitHub bot). Key points:

- **Sandbox (S1).** `DockerSandbox` starts the container with `network_mode="none"`, a memory limit without extra swap, a CPU and process limit, all Linux capabilities dropped and `no-new-privileges`. Files move through the Docker archive API (no host mounts); commands run under `timeout`; a patch that does not apply cleanly leaves the tree untouched.
- **Benchmarks.** QuixBugs is pinned to one upstream commit; its image deletes the reference solutions so the agent cannot read them. SWE-bench Lite uses the official eval scripts and log parsers. A gold-patch sanity check validated 40/40 QuixBugs and 49/50 SWE-bench instances (Chapter 7).
- **Agent (S2).** The model edits with SEARCH/REPLACE blocks; copied line numbers are stripped; Gemma 4's hidden "thinking" is switched off because it consumed the whole output budget in the first runs (decision D10). Every step is logged to a gzipped JSON trajectory.
- **Model server (D17).** Ollama's OpenAI-compatible endpoint silently ignored the per-request context size, so the first B0 runs had a 4,096-token window instead of the configured one; the server's own log showed it (`n_ctx_slot = 4096`). The window is now set in an Ollama model (`gemma4-e4b-32k`: the same weights with a 32,768-token window), `patchpilot eval` checks the server's window before it starts, and the runs whose model calls had filled the window were re-run by a fixed rule (Section 7.5).
- **Localization (A1, A3).** Labels come from parsing the gold patch into edited lines and mapping them to functions with Python's `ast`. Files are represented by a signature skeleton, functions by their source; the same views are built from the sandbox at run time. The cross-encoder is trained on the CPU (`scripts/train_ranker_cpu.py`, D16), and its sequence lengths travel with the model so inference cuts texts exactly as training did.
- **Patch generator (A2).** Training chats are exactly the agent's prompt; the target is the gold fix rewritten as SEARCH/REPLACE blocks, verified to reproduce `git apply`.
- **Evaluation (A4).** Results are written per instance with the git commit; infrastructure stalls are re-run once by a fixed rule (D13); exact McNemar tests compare configurations only when both are complete. `scripts/run_pipeline.ps1` runs every remaining step unattended, one heavy job at a time at below-normal priority, and can be paused and resumed.
- **Failure analysis (A5).** Seven rule categories plus an LLM judge for uncertain cases; gold regions ignore docstrings and blank lines; a blind labelling tool measures agreement.

5.1 Interfaces

The command line runs single fixes and whole experiments and prints any saved trajectory step by step:

patchpilot show bucketsort-0fc99b07

quixbugs/bucketsort (bucketsort-0fc99b07)

#	kind	step	ok	secs	details
0	tool	prepare	yes	0.3	checkout ready
1	tool	run_tests	yes	0.7	{'phase': 'reproduce'} Passed: 1 Failed: 6 failing: python_testcases/test_bucketsort.py::test_bucketsort[input_da… failing: python_testcases/test_bucketsort.py::test_bucketsort[input_da…
2	llm	localize_keywords	yes	123.7	{"keywords": ["bucketsort"], "files": ["python_programs/bucketsort.py"]}
3	tool	search_code	yes	0.2	{'query': 'bucketsort'} python_programs/bucketsort.py:1:def bucketsort(arr, k): python_testcases/test_bucketsort.py:5: from correct_python_programs.bucketsort import bucketsort python_testcases/test_bucketsort.py:7: from python_programs.bucketsort import bucketsort
4	note	localization	yes	0.0	{'localizer': 'llm_search'} python_programs/bucketsort.py
5	tool	read_file	yes	0.1	{'path': 'python_programs/bucketsort.py'} 27 lines
6	llm	repair	yes	162.9	{'attempt': 1} The bug is in how the sorted array is constructed in `bucketsort`. Line 7 iterates over `arr` again, but it should iterate over the `counts` array to build the sorted list based on the counts of each element. python_programs/bucketsort.py
7	tool	edit_file	yes	0.5	{'attempt': 1, 'path': 'python_programs/bucketsort.py', 'search': ' sorted_arr = []\n for i, count in enumerate(arr):\n sorted_arr.extend([i] * count)\n\n', 'replace': ' sorted_arr = []\n for i, count in enumerate(counts):\n sorted_arr.extend([i] * count)\n\n'} edited pytho
8	tool	run_tests	yes	0.5	{'attempt': 1} Passed: 7 Failed: 0 ALL TESTS PASS. ── end of test output ──
9	tool	submit	yes	0.2	{'attempt': 1} diff --git a/python_programs/bucketsort.py b/python_programs/bucketsort.py index 620e64e..b3f453a 100644 ── a/python_programs/bucketsort.py
10	tool	evaluate	yes	0.8	{'attempt': 1} .... [100%] ═══ PASSES ═══ ═══ short test summary info ═══

Result

RESOLVED
attempts used: 1
localized files: python_programs/bucketsort.py
LLM calls: 2, tokens in/out: 4159/171
wall time: 290.77 s

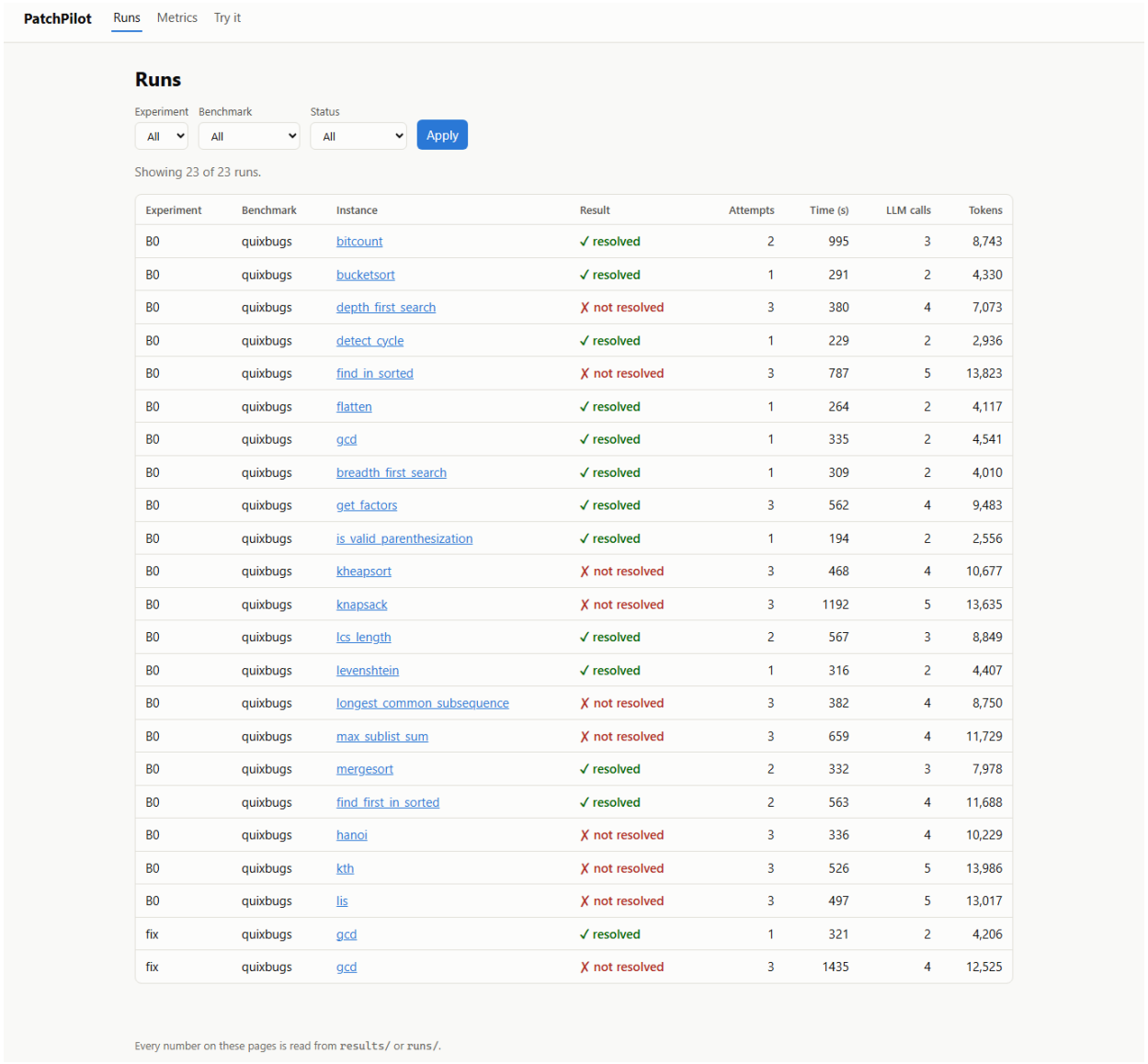
```
diff --git a/python_programs/bucketsort.py b/python_programs/bucketsort.py
index 620e64e..b3f453a 100644
--- a/python_programs/bucketsort.py
+++ b/python_programs/bucketsort.py
@@ -4,7 +4,7 @@ def bucketsort(arr, k):
         counts[x] += 1

     sorted_arr = []
-    for i, count in enumerate(arr):
+    for i, count in enumerate(counts):
         sorted_arr.extend([i] * count)

     return sorted_arr
```

Figure 4. `patchpilot show` on a resolved QuixBugs run

The dashboard (FastAPI, `patchpilot dashboard`) reads `results/` and `runs/`: the list of runs, each trajectory with its final diff, the metrics of every experiment, and a live "Try it" page that runs one fix at a time.



Experiment	Benchmark	Instance	Result	Attempts	Time (s)	LLM calls	Tokens
B0	quixbugs	bitcount	✓ resolved	2	995	3	8,743
B0	quixbugs	bucketsort	✓ resolved	1	291	2	4,330
B0	quixbugs	depth_first_search	✗ not resolved	3	380	4	7,073
B0	quixbugs	detect_cycle	✓ resolved	1	229	2	2,936
B0	quixbugs	find_in_sorted	✗ not resolved	3	787	5	13,823
B0	quixbugs	flatten	✓ resolved	1	264	2	4,117
B0	quixbugs	gcd	✓ resolved	1	335	2	4,541
B0	quixbugs	breadth_first_search	✓ resolved	1	309	2	4,010
B0	quixbugs	get_factors	✓ resolved	3	562	4	9,483
B0	quixbugs	is_valid_parenthesization	✓ resolved	1	194	2	2,556
B0	quixbugs	kheapsort	✗ not resolved	3	468	4	10,677
B0	quixbugs	knapsack	✗ not resolved	3	1192	5	13,635
B0	quixbugs	lcs_length	✓ resolved	2	567	3	8,849
B0	quixbugs	levenshtein	✓ resolved	1	316	2	4,407
B0	quixbugs	longest_common_subsequence	✗ not resolved	3	382	4	8,750
B0	quixbugs	max_sublist_sum	✗ not resolved	3	659	4	11,729
B0	quixbugs	mergesort	✓ resolved	2	332	3	7,978
B0	quixbugs	find_first_in_sorted	✓ resolved	2	563	4	11,688
B0	quixbugs	hanoi	✗ not resolved	3	336	4	10,229
B0	quixbugs	kth	✗ not resolved	3	526	5	13,986
B0	quixbugs	lis	✗ not resolved	3	497	5	13,017
fix	quixbugs	gcd	✓ resolved	1	321	2	4,206
fix	quixbugs	gcd	✗ not resolved	3	1435	4	12,525

Figure 5. Dashboard: experiment runs

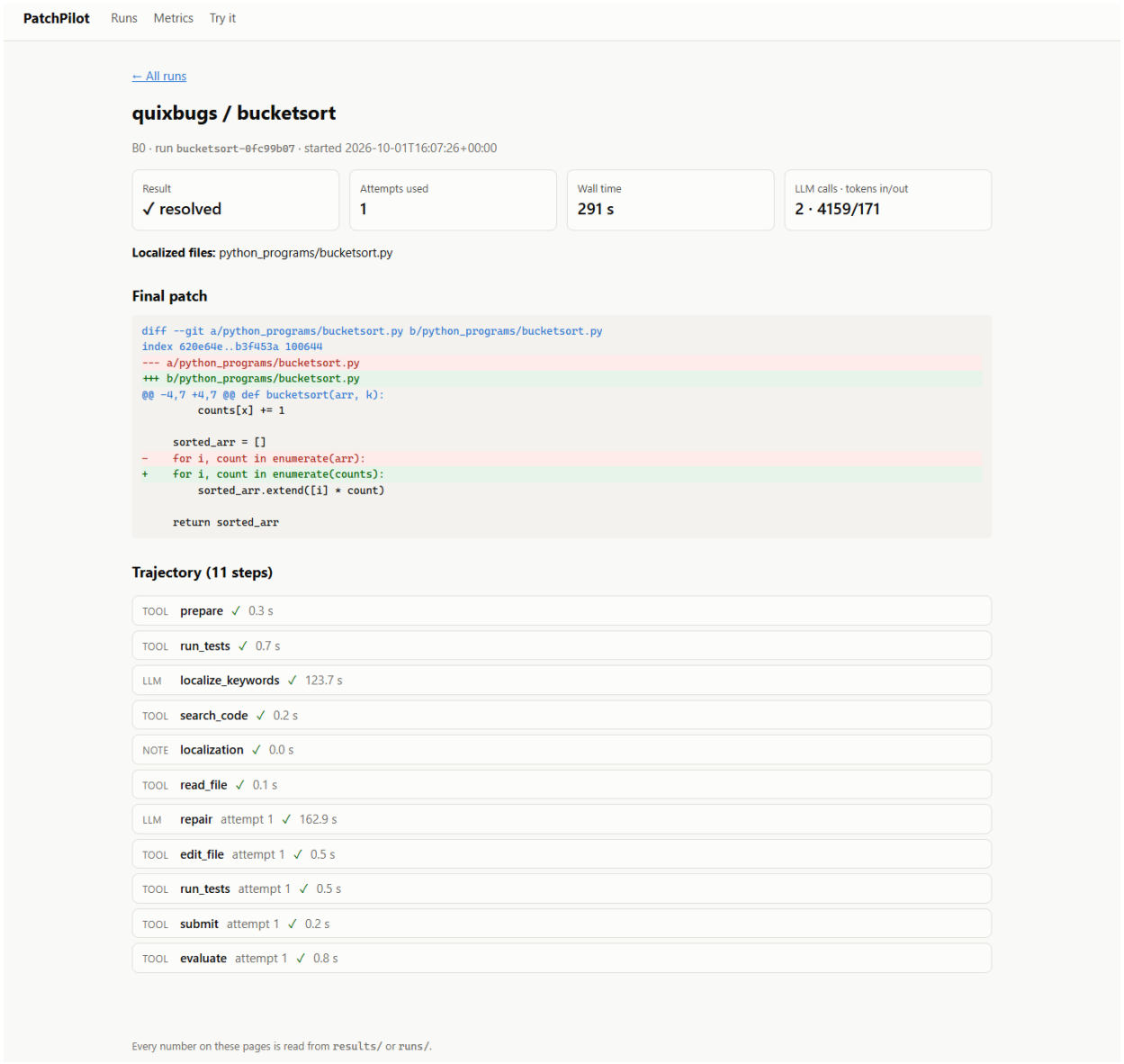


Figure 6. Dashboard: one trajectory with the final patch

PatchPilot Runs Metrics Try it

Metrics

Generated by `scripts/summarize_results.py`, `scripts/classify_failures.py` and `scripts/make_figures.py`.

Resolve rate

Config	Benchmark	Description	Resolved	Rate	95% CI	Status
B0	quixbugs	LLM-search localization, base Gemma 4 E4B, 3 attempts	9/16	56.2%	33–77%	incomplete (16 instances so far)
E1	quixbugs	trained ranker, base Gemma 4 E4B, 3 attempts	-	-	-	TODO: not yet run
E2	quixbugs	LLM-search localization, fine-tuned Gemma 4 E4B, 3 attempts	-	-	-	TODO: not yet run
E3	quixbugs	trained ranker, fine-tuned Gemma 4 E4B, 3 attempts	-	-	-	TODO: not yet run
E4	quixbugs	trained ranker, fine-tuned Gemma 4 E4B, 1 attempt	-	-	-	TODO: not yet run
G1	quixbugs	trained ranker, Gemini API reference (optional)	-	-	-	TODO: not yet run
B0	swebench-lite	LLM-search localization, base Gemma 4 E4B, 3 attempts	-	-	-	TODO: not yet run
E1	swebench-lite	trained ranker, base Gemma 4 E4B, 3 attempts	-	-	-	TODO: not yet run
E2	swebench-lite	LLM-search localization, fine-tuned Gemma 4 E4B, 3 attempts	-	-	-	TODO: not yet run
E3	swebench-lite	trained ranker, fine-tuned Gemma 4 E4B, 3 attempts	-	-	-	TODO: not yet run
E4	swebench-lite	trained ranker, fine-tuned Gemma 4 E4B, 1 attempt	-	-	-	TODO: not yet run
G1	swebench-lite	trained ranker, Gemini API reference (optional)	-	-	-	TODO: not yet run

Paired comparisons (exact McNemar test)

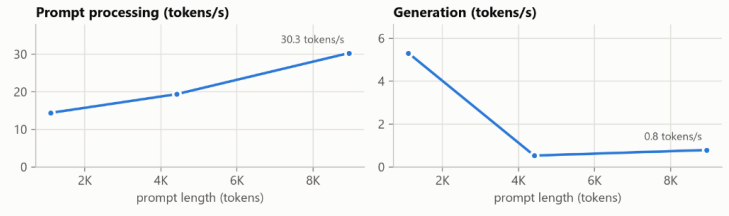
Benchmark	Question	A only	B only	p	Status
quixbugs	RQ2: trained ranker vs LLM search (base model)	-	-	-	TODO: needs both experiments complete
quixbugs	RQ3: fine-tuned vs base model (LLM search)	-	-	-	TODO: needs both experiments complete
quixbugs	RQ3: fine-tuned vs base model (trained ranker)	-	-	-	TODO: needs both experiments complete
quixbugs	RQ1: full system vs baseline	-	-	-	TODO: needs both experiments complete
quixbugs	RQ4: retries with feedback (3 attempts) vs 1 attempt	-	-	-	TODO: needs both experiments complete
swebench-lite	RQ2: trained ranker vs LLM search (base model)	-	-	-	TODO: needs both experiments complete
swebench-lite	RQ3: fine-tuned vs base model (LLM search)	-	-	-	TODO: needs both experiments complete
swebench-lite	RQ3: fine-tuned vs base model (trained ranker)	-	-	-	TODO: needs both experiments complete
swebench-lite	RQ1: full system vs baseline	-	-	-	TODO: needs both experiments complete
swebench-lite	RQ4: retries with feedback (3 attempts) vs 1 attempt	-	-	-	TODO: needs both experiments complete

Failure taxonomy

Experiment	Benchmark	Category	Count	Share
B0 (incomplete)	quixbugs	Patch does not apply / syntax error / no edit produced	3/7	43%
B0 (incomplete)	quixbugs	Right file, wrong location (function or lines)	2/7	29%
B0 (incomplete)	quixbugs	Incomplete fix (right place, tests still fail)	2/7	29%

Figures

gemma4:e4b on the laptop CPU (Ollama)



Source: results/llm_speed_cpu.jsonl

Fault localization on held-out repositories: BM25

Figure 7. Dashboard: metrics

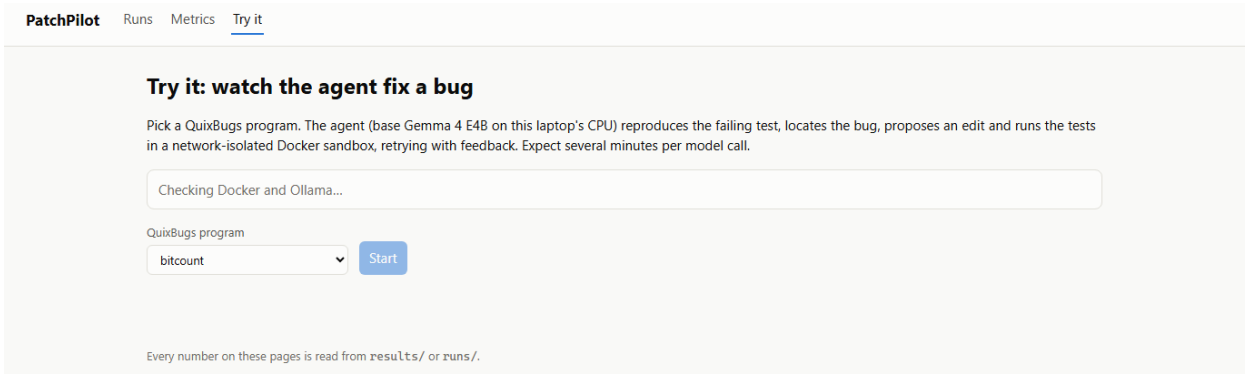


Figure 8. Dashboard: the live "Try it" page

5.2 The GitHub bot

When a pull request's tests fail, the composite action in `github_action/` installs Ollama on the runner, builds a sandbox image from the pull request's tracked files and runs the agent. It only comments; it never pushes or merges.

The bot was tried on a public demo repository: the pull request Refactor basket_total loop introduces an off-by-one bug into a working function. Its test job failed (7 s); the PatchPilot job then ran on a GitHub-hosted runner for 4.2 min (installing Ollama and the model included) and posted the comment shown below: a patch found with `gemma4:e4b` at attempt 1 in 116 s, with the test output before and after. Nothing was pushed or merged.

Source: `results/bot_demo/patchpilot-demo-pr1.json`.

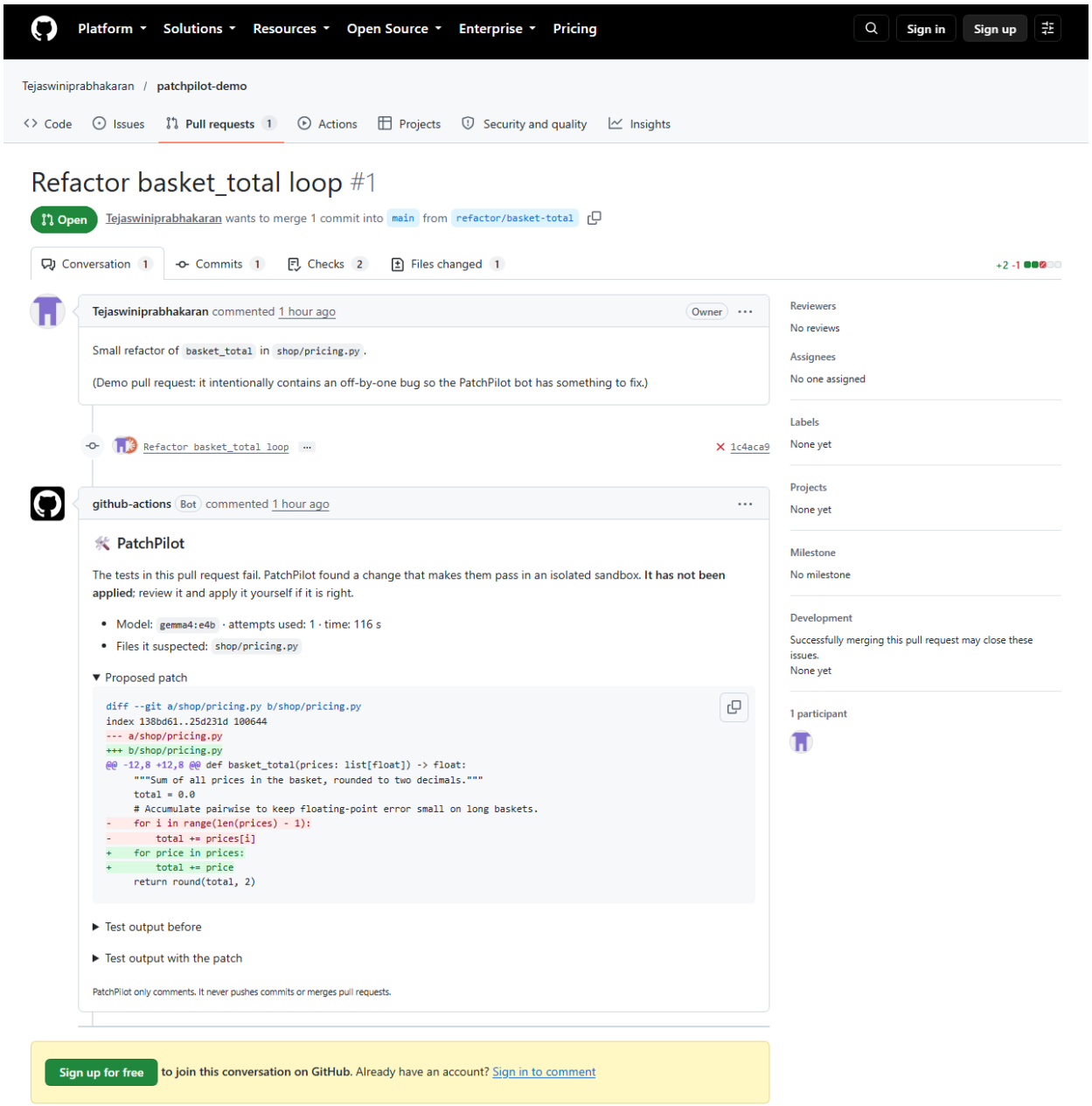


Figure 9. The bot's comment on the demo pull request

Chapter 6 — Datasets and Model Training

6.1 Evaluation data

Set	Size	Source	Notes
QuixBugs (Python)	40 programs	upstream commit `4257f44b`	one-line bugs, reference solutions removed from the image
SWE-bench Lite subset	50 (seed 42), 49 valid	`princeton-nlp/SWE-bench_Lite` revision `6ec7bb89`	`configs/swebench_lite_subset_50.json`
SWE-bench evaluation sample	10 (seed 42)	drawn from the 49 valid	`configs/swebench_lite_subset_10.json`

6.2 Training data and leakage check

Both training sets come from the SWE-bench **training** split, whose 35 repositories are disjoint from SWE-bench Lite's 12. `scripts/check_leakage.py` compared every selected training instance (8,867) and the whole training split with all 300 SWE-bench Lite test instances on repository, instance id, normalised issue text and normalised gold patch: **zero overlap on every key** (`results/leakage_check.json`).

Dataset	Train	Validation	Test	Source file
Localization instances	6,329	1,020	1,212	`results/localization/dataset_stats.json`
Fix-generation chats	3,063	494	580	`results/patchgen/dataset_stats.json`

Splits are by repository (validation: transformers, mmdetection, pip, scipy, fastapi; test: airflow, conda, spaCy, meson). Dataset cards: `docs/dataset_cards/`.

6.3 Training setup

	Fault-localization ranker (trained)	Patch generator (prepared, not trained)
Base model	`microsoft/unixcoder-base` (125M)	`google/gemma-4-E4B-it`
Method	full fine-tuning, 2-class cross-encoder	QLoRA (4-bit NF4, LoRA r=16, α=32, dropout 0.05)
Data	seeded sample of 2,000 training instances; all positives, 4 hard-negative files, 8 negative functions each	3,063 chats
Length	256 tokens (bug report: 128)	3,072 tokens
Optimisation	lr 3e-5, batch 16, at most 1 epoch within a 4-hour budget, fp32	lr 2e-4 cosine, batch 16, 1 epoch, paged 8-bit AdamW
Hardware	laptop CPU (D16)	Kaggle T4 (notebook 02, needs the owner's account)

The full-scale GPU recipe for the ranker (all 113,396 pairs, 384 tokens) stays in notebook 01; the CPU run below is the model used in every reported result.

6.4 Ranker training run

CPU training run of the ranker (D16): TODO: not yet run.

Chapter 7 — Results and Discussion

All tables state their source; `TODO: not yet run` marks results that do not exist yet. The result sections are regenerated from `results/` every time this report is built.

7.1 Benchmark validity (`results/sanity_summary.csv`)

Benchmark	Instances	Fail when buggy	Pass with gold patch	Valid
QuixBugs	40	40	40	40
SWE-bench Lite subset	50	50	49	49

`psf__requests-1963` is excluded: 50 of its tests contact an external HTTP server and the sandbox has no network by design (D11).

7.2 Inference cost on the laptop (`results/llm_speed_cpu.jsonl`)

Gemma 4 E4B processed 14.4–30.3 prompt tokens/s and generated 0.5–5.3 tokens/s for prompts of 1,102–8,950 tokens (127–712 s per call), which is why experiments run at a small scale (D12).

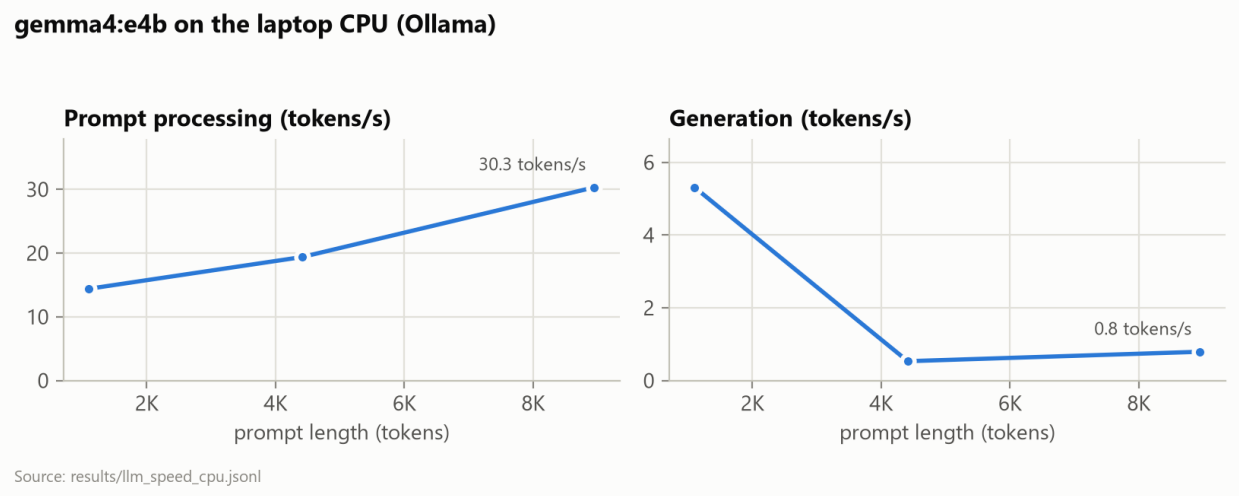


Figure 10. Gemma 4 E4B speed on the laptop CPU

7.3 Fault localization (RQ2, part 1)

Held-out comparison of BM25, embeddings and the trained ranker: TODO: not yet run.

All 1,212 held-out test instances, BM25 only (reference for the sample above):

Method	File Top-1	File Top-5	Func. Top-1	Func. Top-3	Func. Top-5	Func. MRR
BM25	52.1%	98.1%	25.8%	43.0%	52.9%	0.390

Source: `results/localization/heldout_bm25_test.json`.

Whole-repository localization on the SWE-bench subset: TODO: not yet run.

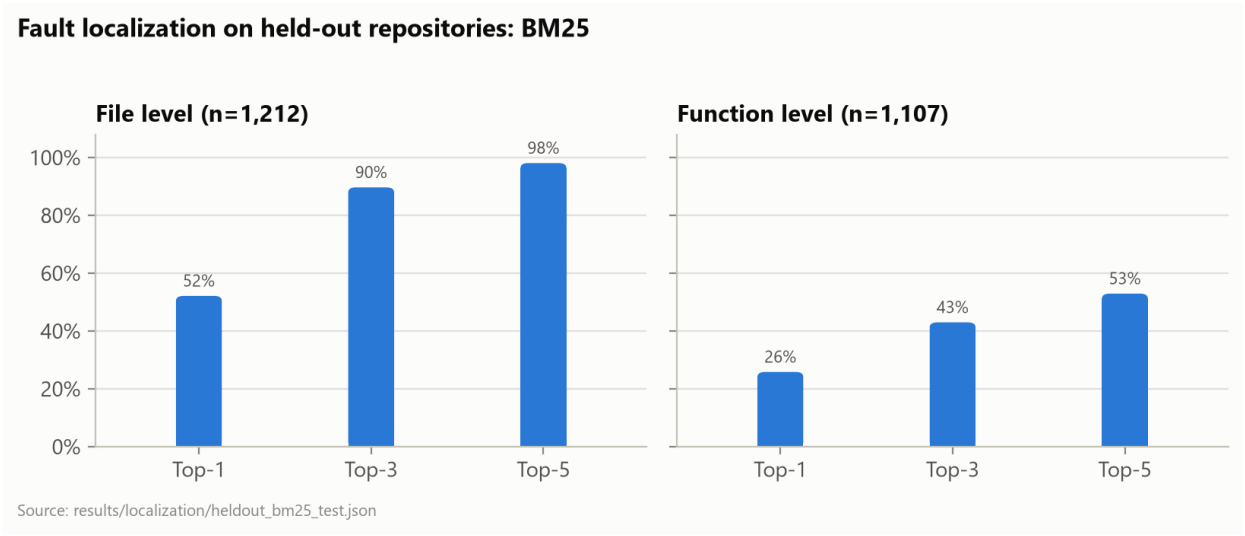


Figure 11. Top-k fault-localization accuracy on held-out repositories

File-level Top-5 on the held-out set is close to trivial (about five candidates per instance, the gold file always present); the function level (about 112 candidates) is the informative comparison. The whole-repository setting on the SWE-bench subset is the one the agent faces.

7.4 Resolve rates and paired tests (RQ1, RQ2, RQ4)

Config	Description	QuixBugs	SWE-bench Lite subset
B0	LLM-search localization, base Gemma 4 E4B, 3 attempts	9/16 (56.2%, 95% CI 33.2% to 76.9%; run still in progress)	TODO: not yet run
E1	trained ranker, base Gemma 4 E4B, 3 attempts	TODO: not yet run	TODO: not yet run
E2	LLM-search localization, fine-tuned Gemma 4 E4B, 3 attempts	TODO: not yet run	TODO: not yet run
E3	trained ranker, fine-tuned Gemma 4 E4B, 3 attempts	TODO: not yet run	TODO: not yet run
E4	trained ranker, fine-tuned Gemma 4 E4B, 1 attempt	TODO: not yet run	TODO: not yet run
G1	trained ranker, Gemini API reference (optional)	TODO: not yet run	TODO: not yet run

Source: `results/summary/resolve_rates.csv`.

Paired comparisons (exact McNemar test on the instances both runs attempted; TODO: a run is missing or incomplete):

Question	Benchmark	A vs B	A resolved	B resolved	A only	B only	p
RQ2: trained ranker vs LLM search (base model)	QuixBugs	B0 vs E1	-	-	-	-	TODO
RQ3: fine-tuned vs base model (LLM search)	QuixBugs	B0 vs E2	-	-	-	-	TODO
RQ3: fine-tuned vs base model (trained ranker)	QuixBugs	E1 vs E3	-	-	-	-	TODO
RQ1: full	QuixBugs	B0 vs E3	-	-	-	-	TODO

system vs baseline							
RQ4: retries with feedback (3 attempts) vs 1 attempt	QuixBugs	E4 vs E3	-	-	-	-	TODO
RQ2: trained ranker vs LLM search (base model)	SWE-bench Lite subset	B0 vs E1	-	-	-	-	TODO
RQ3: fine-tuned vs base model (LLM search)	SWE-bench Lite subset	B0 vs E2	-	-	-	-	TODO
RQ3: fine-tuned vs base model (trained ranker)	SWE-bench Lite subset	E1 vs E3	-	-	-	-	TODO
RQ1: full system vs baseline	SWE-bench Lite subset	B0 vs E3	-	-	-	-	TODO
RQ4: retries with feedback (3 attempts) vs 1 attempt	SWE-bench Lite subset	E4 vs E3	-	-	-	-	TODO

Source: `results/summary/paired_tests.csv`.

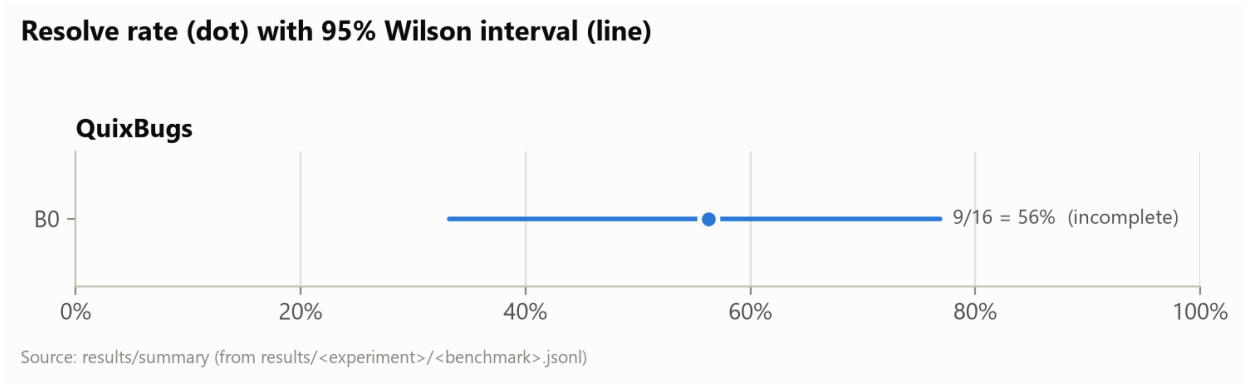


Figure 12. Resolve rate per configuration with 95% Wilson intervals

B0@1 and E1@1 are B0 and E1 counted as if they had stopped after the first attempt (from the same runs), which measures what the retries add.

7.5 Runs re-run by fixed rules (D13, D17)

A run is re-run once if a model call stalled for more than 30 minutes, the model server failed, or a model call filled the server's context window. These rules ignore whether the bug was fixed, so re-running cannot favour any configuration.

Experiment	Instance	Reason (fixed rule)	First run	Re-run
B0	breadth_first_search	model call took 3799 s (> 1800 s)	not resolved	resolved

Source: `results/B0/infra_failures.jsonl`, `results/B0/<benchmark>.jsonl`.

7.6 Cost per instance

Config	Benchmark	Minutes (mean / median)	Model calls	Prompt tokens	Output tokens	Attempts	Status
B0	QuixBugs	9.4 / 8.6	3.4	7,575	988	2.12	incomplete (16 instances so far)

Source: `results/summary/cost.csv`.

7.7 Failure analysis (RQ5)

Experiment	Benchmark	Category	Meaning	Runs	Share
B0	QuixBugs	patch invalid	Patch does not apply / syntax error / no edit produced	3 of 7	42.9%
B0	QuixBugs	wrong function	Right file, wrong location (function or lines)	2 of 7	28.6%
B0	QuixBugs	incomplete fix	Incomplete fix (right place, tests still fail)	2 of 7	28.6%

Source: `results/analysis/taxonomy.csv`, `results/analysis/failure_labels.jsonl`.

Agreement with manual labels: TODO: the blind labelling tool (`scripts/label_failures.py`) needs a person.

7.8 Answers to the research questions

- **RQ1 (how many real bugs can an open-weight agent fix?)** With base Gemma 4 E4B on a laptop CPU: B0 9/16 (56.2%, 95% CI 33.2% to 76.9%; run still in progress) on QuixBugs. The planned comparison of the full system (E3, with the fine-tuned patch generator) against the baseline is not available, because the fine-tuned model was not trained.
- **RQ2 (does the trained ranker help?)** QuixBugs: TODO: not yet run (needs complete B0 and E1 runs). SWE-bench Lite subset: TODO: not yet run (needs complete B0 and E1 runs).
- **RQ3 (does fine-tuning help?)** Not answered: E2-E4 need the fine-tuned model (notebook 02 and its dataset are ready).
- **RQ4 (do retries with test feedback help?)** QuixBugs: TODO: not yet run (needs complete B0@1 and B0 runs). SWE-bench Lite subset: TODO: not yet run (needs complete B0@1 and B0 runs). QuixBugs: TODO: not yet run (needs complete E1@1 and E1 runs). SWE-bench Lite subset: TODO: not yet run (needs complete E1@1 and E1 runs).
- **RQ5 (why does the agent fail?)** Most common category: B0 on QuixBugs: patch invalid (3 of 7 failed runs).

Answers marked TODO are filled in automatically when `scripts/run_pipeline.ps1` finishes and this report is rebuilt.

7.9 Threats to validity

- **Small samples.** Ten SWE-bench instances cannot show small differences; paired tests are reported with their p-values and no claim is made without one.
- **One run per configuration.** Temperature 0 and a fixed seed make runs repeatable, but a different seed or model version could change individual outcomes.
- **Reduced-scale ranker.** The cross-encoder was trained on a CPU with a sample of the training data and shorter sequences (D16); the full-scale recipe might rank better.

- **The agent sees the failing test** (D8), so SWE-bench numbers are not comparable with the public leaderboard.
- **Infrastructure.** A context-window defect affected early B0 runs; it was found, fixed and the affected runs re-run by a fixed rule (D17). Timings come from a laptop that was also used for other work.

Chapter 8 — Testing

The suite has 182 unit tests and 4 integration tests (counted by pytest when this report was built).

Unit tests (pytest) cover the sandbox (with a mocked Docker client), benchmarks, tools, the agent loop (scripted model, in-memory sandbox), the model client and its context-window check, localization and ranker training helpers, patch-generation data, statistics, failure analysis, the CLI, the dashboard (FastAPI test client with a fake live runner) and the bot (faked Docker, model and agent). The integration tests run a real QuixBugs container and check that the network is unreachable, runaway commands are killed and rejected patches leave the tree untouched. GitHub Actions runs ruff, a format check, strict mypy and the unit tests on every push, and every merged pull request passed CI. The training and evaluation scripts were also smoke-tested end to end with a tiny stand-in model before the real runs.

Chapter 9 — Conclusion and Future Work

PatchPilot delivers a complete, tested repair system — sandbox, agent, CLI, dashboard and GitHub bot — together with leakage-checked training data, a fault-localization ranker trained on a laptop CPU, and an evaluation design that isolates each component's contribution with paired tests. Chapter 7 answers the questions the available runs can answer (RQ1, RQ2, RQ4, RQ5) directly from `results/`; the questions about fine-tuning wait for the GPU run that notebook 02 prepares.

Limitations. A small model on a CPU; a small SWE-bench sample, so most differences there will not be statistically significant; a reduced-scale ranker; the agent sees the failing test, so SWE-bench numbers are not comparable with the public leaderboard (D8); Python only.

Future work. Training the QLoRA patch generator (notebook 02) and running E2–E4; the full-scale ranker; reproduction-test generation so the agent can work without a given failing test; the full 300-instance SWE-bench Lite; Java via Defects4J [10].

References

[1] C. Le Goues, T. Nguyen, S. Forrest and W. Weimer, "GenProg: A generic method for automatic software repair," *IEEE Transactions on Software Engineering*, vol. 38, no. 1, pp. 54–72, 2012.

[2] Z. Chen, S. Kommrusch, M. Tufano, L.-N. Pouchet, D. Poshyvanyk and M. Monperrus, "SequenceR: Sequence-to-sequence learning for end-to-end program repair," *IEEE Transactions on Software Engineering*, 2019, doi: 10.1109/TSE.2019.2940179.

[3] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Proc. ESEC/FSE*, 2022, pp. 959–971.

[4] C. S. Xia and L. Zhang, "Keep the conversation going: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT," in *Proc. ISSTA*, 2024, arXiv:2304.00385.

[5] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?," in *Proc. ICLR*, 2024, arXiv:2310.06770.

[6] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[7] Y. Zhang, H. Ruan, Z. Fan and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proc. ISSTA*, 2024, arXiv:2404.05427.

[8] C. S. Xia, Y. Deng, S. Dunn and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," in *Proc. FSE*, 2025, arXiv:2407.01489.

[9] D. Lin, J. Koppel, A. Chen and A. Solar-Lezama, "QuixBugs: A multi-lingual program repair benchmark set based on the Quixey Challenge," in *Proc. SPLASH Companion*, 2017, pp. 55–56, doi: 10.1145/3135932.3135941.

[10] R. Just, D. Jalali and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for Java programs," in *Proc. ISSTA*, 2014, pp. 437–440.

[11] R. Abreu, P. Zoeteweij and A. J. C. van Gemund, "On the accuracy of spectrum-based fault localization," in *Proc. TAIC PART*, 2007, pp. 89–98.

[12] X. Li, W. Li, Y. Zhang and L. Zhang, "DeepFL: Integrating multiple fault diagnosis dimensions for deep fault localization," in *Proc. ISSTA*, 2019.

[13] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of EMNLP*, 2020, arXiv:2002.08155.

[14] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proc. ACL*, 2022, arXiv:2203.03850.

[15] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022, arXiv:2106.09685.

[16] T. Dettmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2305.14314.

[17] J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[18] Hugging Face, "Welcome Gemma 4: Frontier multimodal intelligence on device," 2026. [Online]. Available: https://huggingface.co/blog/gemma4

[19] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[20] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[21] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

Appendix

- **A. Sample trajectory:** `patchpilot show <run_id>` or the dashboard; raw files in `results/B0/trajectories/`.
- **B. Configuration files:** `configs/exp_baseline.yaml`, `configs/exp_e1_ranker.yaml`, `configs/agent_dev.yaml`, the seeded subsets in `configs/` and the Ollama model in `configs/ollama/`.
- **C. User manual:** the README's Quickstart, "Reproducing the experiments" and "GitHub bot".
- **D. Running the remaining experiments:** `powershell -File scripts/run_pipeline.ps1 -Detach` (progress in `logs/pipeline.log`).
- **E. Rebuilding this report:** `python scripts/capture_figures.py` (screenshots and diagrams), `python scripts/make_figures.py` (charts), then `python scripts/build_report.py`.